

RAPPORT TECHNIQUE — PLATEFORME DFIR & THREAT HUNTING

Documentation de l'architecture d'analyse forensique et de réponse aux incidents

Membres du projet :	Feres Elloumi & Mohamed Smadhi	Encadrante :	Mme Maissa
Version du système :	v1.0.0 (Opérationnel)	Statut global :	Tests validés — Prêt pour démonstration

PROCESSUS ANALYSÉS
209 Éléments

ALERTE DE SÉCURITÉ
5 Détections Actives

1. Présentation de l'Architecture Technique

Notre solution s'appuie sur une boucle d'inspection continue. Un collecteur (`collector.py`) extrait la télémétrie des processus en cours d'exécution et identifie les comportements anormaux. Les données sont traitées par une API REST développée avec FastAPI (`main.py`) et restituées en temps réel sur un tableau de bord web interactif.

2. Télémétrie et Actions Réponse (Incident Response)

NIVEAU	PROCESSUS	PID	COMPTE UTILISATEUR	ACTION DISPONIBLE
ÉLEVÉ	cmd.exe	1164	DESKTOP-07QC65B\dell	Terminer (Kill)
ÉLEVÉ	cmd.exe	2704	DESKTOP-07QC65B\dell	Terminer (Kill)
ÉLEVÉ	powershell.exe	7256	DESKTOP-07QC65B\dell	Terminer (Kill)
ÉLEVÉ	powershell.exe	7488	DESKTOP-07QC65B\dell	Terminer (Kill)
ÉLEVÉ	cmd.exe	12916	DESKTOP-07QC65B\dell	Terminer (Kill)

3. Moteur de Détection YARA (Custom Rules Engine)

Afin d'éviter la dépendance aux compilateurs C++ sous Windows, nous avons conçu un moteur d'analyse natif en Python (`yara_scanner.py`). Il charge les signatures définies dans `yara_rules.json`, calcule l'empreinte numérique SHA-256 du fichier analysé et recherche la présence d'expressions ou de chaînes malveillantes.

Exemple de résultat d'analyse YARA :

```
{
  "filepath": "test_malware.txt",
  "sha256": "e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855",
  "matches_found": 1,
  "matches": [{"rule_name": "Webshell_Or_ReverseShell_Detect", "severity": "CRITICAL"}]
}
```